

Building a Zenoh-based Robotics Stack on a Raspberry Pi



“Dash” our 2024 *Rival Kickstart* competition robot

In competitive robotics, having a simple yet powerful software stack that handles real-time communication and control is crucial. For our 2024 Rival Kickstart competition robot, "Dash," we selected Zenoh: a zero-overhead pub/sub (publish/subscribe) framework designed specifically for robotics to streamline our robot's networking and control systems. Combined with the high-performance MJBots moteus motor controllers and intuitive visualization tools provided by Rerun.io, this Zenoh-based stack enabled rapid development, robust communication, and efficient debugging, significantly enhancing our ability to quickly iterate and deploy reliable robot systems on a Raspberry Pi.

You can reference our robot code here: <https://github.com/BenCaunt/dash-robotcode>

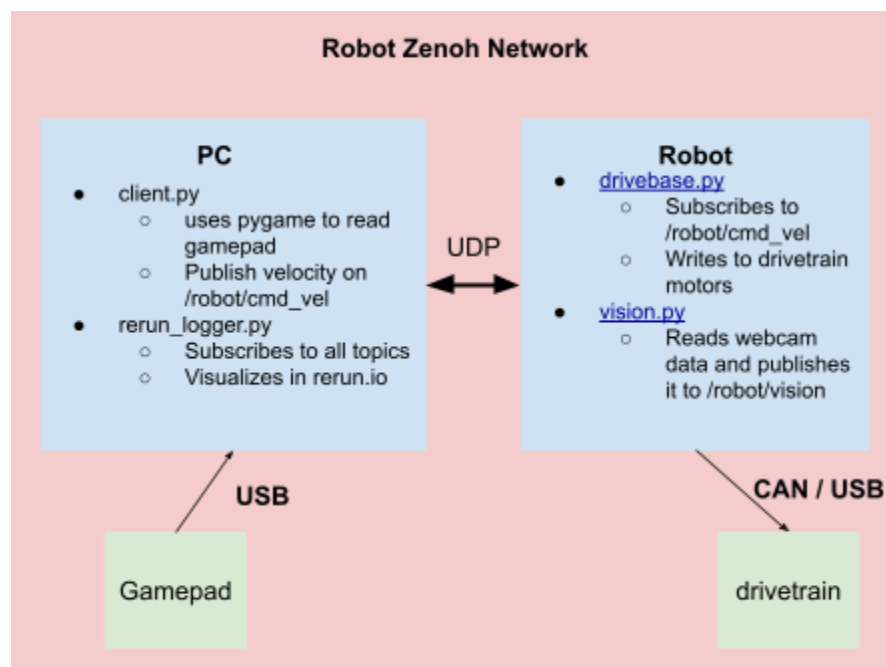
Zenoh (<https://zenoh.io/>) is a self proclaimed “Zero overhead pub/sub protocol” designed for fast network based communication for robotics. It has several features that make it a great choice:

- Peer-to-peer by default: every process that links the Zenoh runtime can publish/subscribe without a central broker.
- Auto-discovery: a tiny “scouting” message on the local UDP-multicast group tells new peers who is already present, so the Pi and your laptop see each

other as soon as they share a LAN or Wi-Fi network. This means that by default you never need to configure IP addresses.

- Bridges: later you can drop in zenoh-bridge-ros2 or zenoh-pico to talk to ROS 2 nodes or micro-controllers. This is useful if you want to use the extensive library of ROS 2 packages without committing yourself to that ecosystem.
- Multiple Language Support: Zenoh is written in Rust enabling a high degree of robustness and performance. It also has easy to use bindings in Python and C++ meaning you can use familiar languages.
- Super simple quickstart: You can have two devices communicating in less than 20 lines of code. See the Zenoh docs here: <https://zenoh.io/docs/getting-started/first-app/>
- Cross platform: Unlike ROS, Zenoh is very friendly with different operating systems. In our setup we run our driver station code on a MacBook and robot code on raspbian but any modern OS should work just fine.

Robotics Network Topology



Device	What runs on it	Purpose
Laptop (PC)	client.py – reads a USB game-pad with pygame and publishes drive commands on /robot/cmd_vel. rerun_logger.py – subscribes to every topic and streams data into the Rerun visualizer.	Operator interface + logging
Robot (Raspberry Pi / Jetson)	drivebase.py – subscribes to /robot/cmd_vel and drives the motors via CAN or USB. vision.py – captures webcam frames and publishes them on /robot/vision.	Real-time actuation + sensing

- Both devices are on the same Wi-Fi. Zenoh’s **auto-discovery** (“**scout**”) **messages** let them find each other instantly without any additional configuration
- Topics are independent of the processes that use them. You can **start, stop, or add new scripts** (e.g. **arm.py**, **shooter.py**) without touching the existing ones.
- Any script can subscribe to **/robot/vision** (or any other key) and immediately receive data published by others. For example, you could extend **client.py** to listen to **/robot/vision**, detect AprilTags, and send corrected drive commands for auto-aim—leveraging Zenoh’s millisecond-level latency.

Zenoh gives you a plug-and-play data bus: drop in new capabilities, reuse sensor streams, and keep control code loosely coupled—all with minimal networking boilerplate.

Logging and Visualization

For logging and visualization we use rerun.io (<http://rerun.io>) which is an excellent tool for robotics and multimodal data visualization in general. It is natively written in

Rust and has easy to use Python APIs and excellent documentation. For performance, we usually setup a script that subscribes to Zenoh topics on our client and does all rerun logging on the PC. But, Rerun does natively support network logging (though it uses TCP which is less desirable for a competition setting where hostile network conditions may be present)

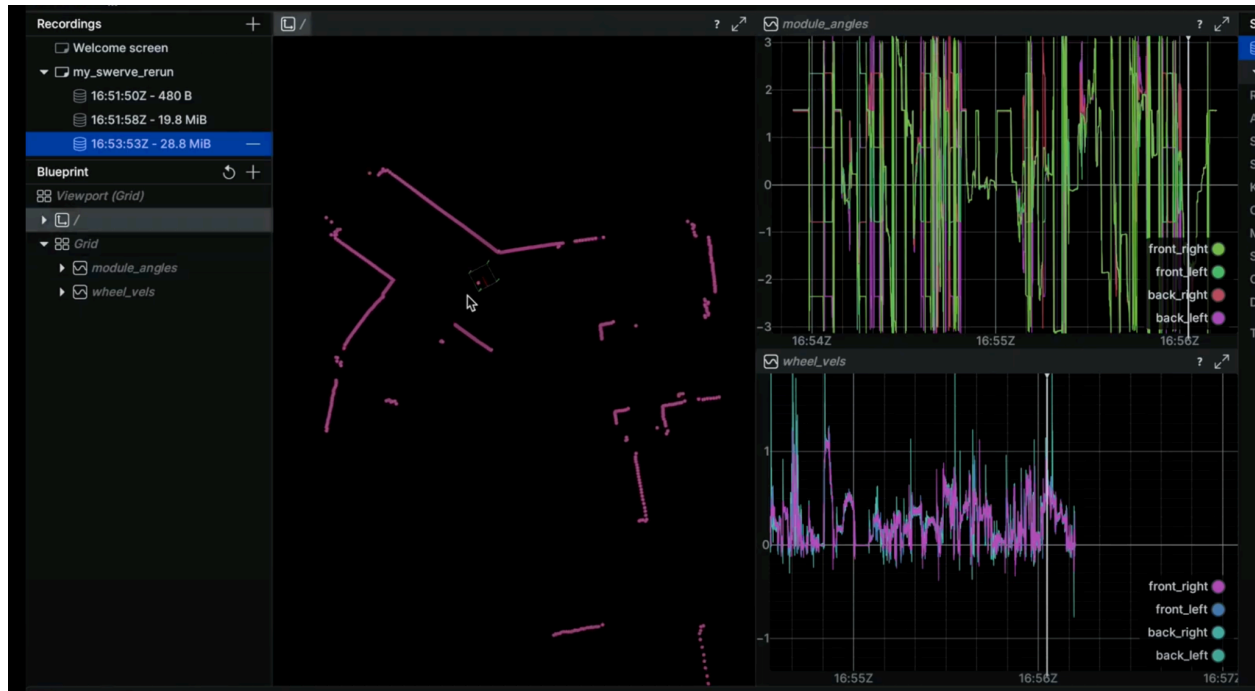
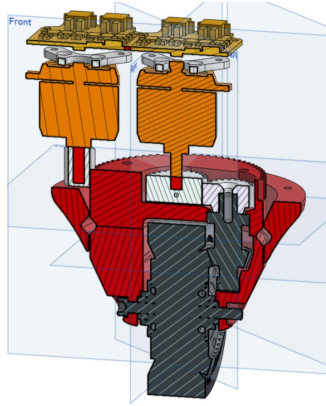


Image depicts a Rerun visualization of our Rival kickstart robot during offseason LiDAR testing. Left shows robot and LiDAR data being visualized, right shows basic telemetry like swerve module angles and wheel velocities.

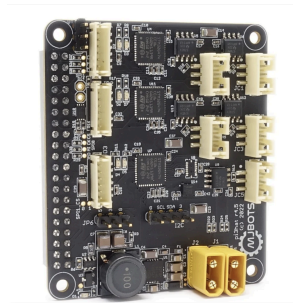
MJBots Moteus Motor Controllers

We use the MJBots [moteus r4.11 motor controllers](#). These are closed loop brushless motor controllers that attach to the back of any hobby brushless motor and turn it into a high torque brushless servo that can be controlled over CAN.

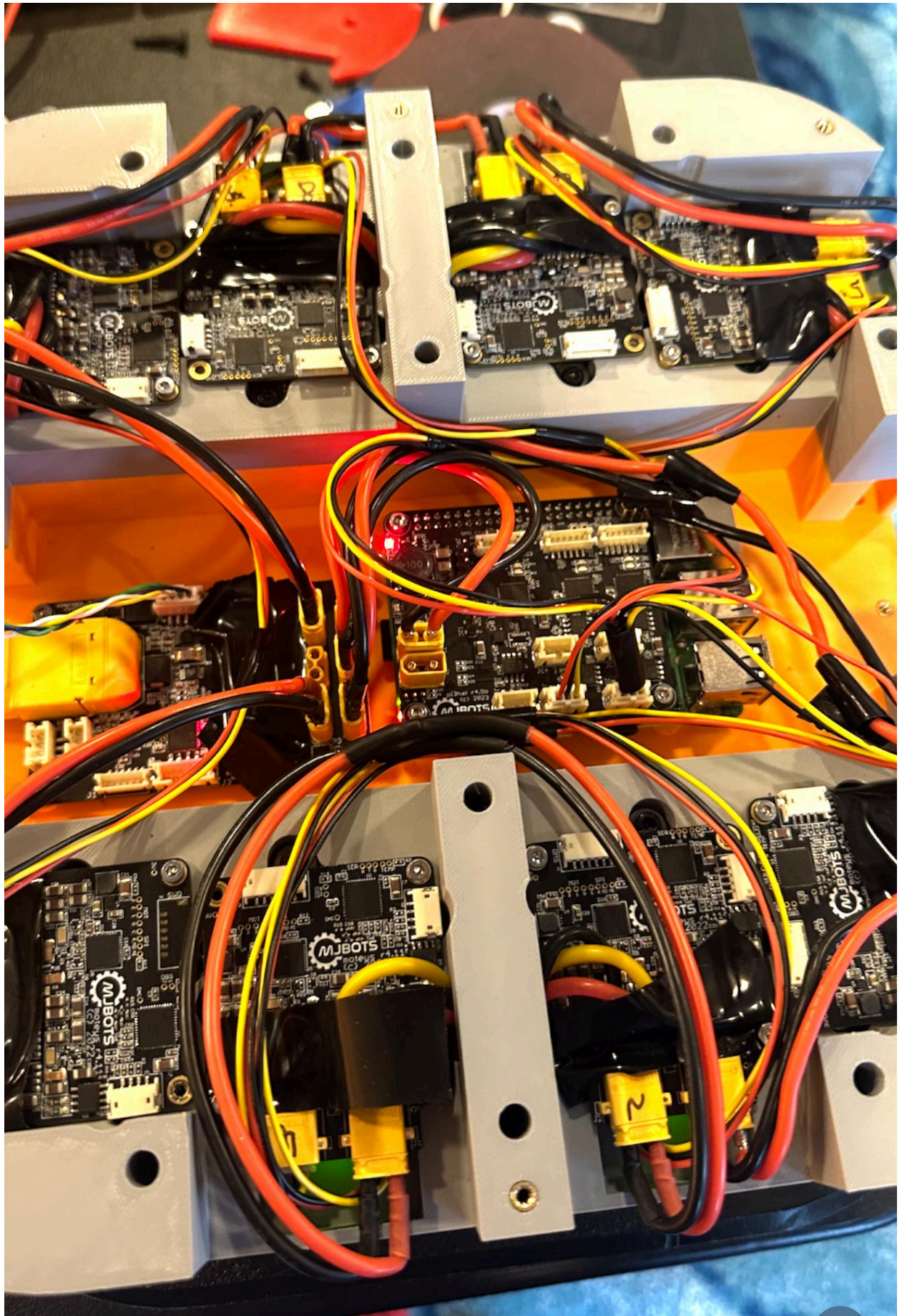


Swerve module CAD featuring moteus controller (yellow) and off the shelf brushless motors (orange).

The Moteus control system enables high performance closed-loop FOC brushless motor control at a relatively low cost. They have great documentation on [Github](https://github.com/mjbotsworld/mjbotsworld) and we've had an excellent support experience on the Mjbots discord server. These motor controllers use a CAN Bus meaning a single wire can daisy chain through the whole robot which is particularly nice for complex robots like swerve as it significantly cuts down on total wire runs. MJBots provides good software support for their python and C++ libraries as well as providing a custom T-View application for visual tuning. One thing that was particularly useful was their raspberry pi hat (note it is only compatible with the Pi 3 and 4, not the Pi 5). This hat comes with a built in gyroscope for robot orientation estimation.



MJBots raspberry pi hat: <https://mjbots.com/products/mjbots-pi3hat-r4-5>



Moteus controllers on Dash

Conclusion / TLDR

Zenoh provides an efficient, zero-overhead pub/sub networking stack ideal for robotics, particularly when running on resource-constrained devices like a Raspberry Pi. Its peer-to-peer communication, automatic device discovery, and flexible bridging options (e.g., ROS 2 interoperability) simplify robot system integration while significantly reducing network setup complexity. Combined with robust visualization tools like Rerun.io, Zenoh enables rapid development and debugging workflows. Coupled with MJBots moteus motor controllers, which provide easy-to-integrate high-performance brushless motor control over CAN bus, this stack forms a powerful, cost-effective solution for competitive robotics applications like our 2024 Rival Kickstart competition robot, "Dash."

Key Resources:

- Zenoh: <https://zenoh.io>
- Rerun visualization: <https://rerun.io>
- MJBots moteus controllers: <https://mjbots.com/products/moteus-r4-11>
- Dash robot code: <https://github.com/BenCaunt/dash-robotcode>